

Guida ai Comandi SQL e alle Transazioni

Questa lezione raccoglie i principali comandi SQL in una visione operativa. L'obiettivo non e' memorizzare una lista di parole chiave, ma capire come SQL consente di:

- definire e modificare schemi
- inserire, leggere, aggiornare e cancellare dati
- ricostruire informazioni distribuite tra tabelle
- filtrare risultati
- usare query innestate
- proteggere operazioni complesse tramite transazioni

Gli esempi usano due scenari gia' noti:

- rubrica e contatti nella sandbox base
- palestra, schede ed esecuzioni nella sandbox di progetto

1. Famiglie di comandi SQL

Una classificazione utile distingue:

- comandi per lo schema
- comandi per i dati
- comandi per il controllo delle transazioni

Nel corso li leggeremo sempre rispetto al modello relazionale:

- lo schema descrive relazioni, attributi e vincoli
- i dati sono tuple dentro relazioni
- le query producono nuove viste sui dati
- le transazioni proteggono passaggi di stato complessi

2. CRUD sullo schema

Quando lavoriamo sullo schema, le operazioni CRUD non riguardano righe ma strutture.

Create: CREATE TABLE

Esempio:

```
CREATE TABLE NoteContatto (  
    id_nota INTEGER PRIMARY KEY,  
    id_contatto INTEGER NOT NULL,  
    testo TEXT NOT NULL,  
    data_nota TEXT NOT NULL,  
    FOREIGN KEY (id_contatto) REFERENCES Contatti(id)  
);
```

Qui stiamo creando una nuova relazione. La tabella non e' solo un contenitore: include gia' alcune regole.

Da osservare:

- `id_nota` identifica la nota
- `id_contatto` collega la nota a un contatto
- `NOT NULL` impone attributi obbligatori
- `FOREIGN KEY` impedisce note riferite a contatti inesistenti

Read: leggere lo schema

In SQLite, per ispezionare lo schema si puo' interrogare il catalogo interno:

```
SELECT name
FROM sqlite_master
WHERE type = 'table'
ORDER BY name;
```

In altri DBMS esistono viste di sistema analoghe, spesso sotto `information_schema`.

Update: ALTER TABLE

Modificare una tabella esistente richiede cautela.

Esempio:

```
ALTER TABLE Contatti
ADD COLUMN email TEXT;
```

Questa istruzione cambia la struttura della tabella. Il punto importante e':

- cosa succede ai dati gia' presenti?
- il nuovo attributo puo' essere nullo?
- quali applicazioni dipendono dallo schema precedente?

Delete: DROP TABLE

Esempio:

```
DROP TABLE NoteContatto;
```

Questa istruzione elimina la struttura e i dati della tabella.

Prima di usarla bisogna chiedersi:

- la tabella e' referenziata da altre tabelle?
- contiene dati da conservare?
- e' possibile ricostruirla?

3. CRUD sui dati

Quando lavoriamo sui dati, le CRUD agiscono sulle tuple.

Create: INSERT

```
INSERT INTO Contatti (id, nome, cognome, telefono)
VALUES (6, 'Elena', 'Bruni', '06-0000000');
```

Lettura:

- creo una nuova tupla
- assegno valori agli attributi indicati
- devo rispettare chiavi e vincoli

Read: SELECT

```
SELECT id, nome, cognome, telefono
FROM Contatti
ORDER BY cognome, nome;
```

Lettura:

- FROM indica da dove leggo
- SELECT indica quali colonne voglio vedere
- ORDER BY ordina il risultato

Update: UPDATE

```
UPDATE Contatti
SET telefono = '06-1111111'
WHERE id = 6;
```

Il punto critico e' WHERE.

Senza WHERE, la modifica riguarda tutte le righe:

```
UPDATE Contatti
SET telefono = '06-1111111';
```

Questa query e' sintatticamente corretta ma quasi sempre sbagliata dal punto di vista applicativo.

Delete: DELETE

```
DELETE FROM Contatti
WHERE id = 6;
```

Anche qui il filtro e' essenziale.

Domanda guida:

Sto cancellando esattamente le righe che intendo cancellare?

4. Filtri con WHERE

WHERE seleziona righe.

Esempi:

```
SELECT nome, cognome
FROM Contatti
WHERE telefono IS NULL;

SELECT nome, cognome, telefono
FROM Contatti
WHERE telefono LIKE '06%';

SELECT nome, cognome, telefono
FROM Contatti
WHERE telefono LIKE '06%'
    OR telefono IS NULL;
```

Punti importanti:

- IS NULL non e' uguale a = NULL
- LIKE serve per pattern testuali
- AND, OR, NOT combinano condizioni

5. JOIN: ricostruire informazioni distribuite

Nel modello relazionale, i dati sono distribuiti tra tabelle collegate.

Nel dominio palestra:

- una scheda appartiene a un iscritto
- una scheda e' creata da un istruttore
- una scheda contiene piu' esercizi
- le esecuzioni registrano eventi reali

INNER JOIN

```
SELECT i.nome, i.cognome, s.titolo
FROM SchedeAllenamento s
JOIN Iscritti i ON s.id_iscritto = i.id_iscritto;
```

Tiene solo le righe che hanno corrispondenza.

LEFT JOIN

```
SELECT i.nome, i.cognome, COUNT(ex.id_esecuzione) AS numero_esecuzioni
FROM Iscritti i
LEFT JOIN Esecuzioni ex ON i.id_iscritto = ex.id_iscritto
GROUP BY i.id_iscritto, i.nome, i.cognome;
```

Mantiene tutti gli iscritti, anche quelli senza esecuzioni.

Questa differenza e' centrale:

- JOIN mostra solo cio' che ha match
- LEFT JOIN preserva il lato sinistro

6. JOIN multipli

Per ricostruire il dettaglio di una scheda servono piu' tabelle:

```
SELECT
    i.nome,
    i.cognome,
    s.titolo,
    e.nome_esercizio,
    se.serie,
    se.ripetizioni
FROM SchedeAllenamento s
JOIN Iscritti i ON s.id_iscritto = i.id_iscritto
JOIN SchedaEsercizi se ON s.id_scheda = se.id_scheda
JOIN Esercizi e ON se.id_esercizio = e.id_esercizio
WHERE s.attiva = 1
ORDER BY i.cognome, i.nome, se.ordine_esecuzione;
```

Lettura progressiva:

1. parto dalle schede
2. recupero l'iscritto
3. recupero le righe della scheda
4. recupero il nome degli esercizi
5. filtro le schede attive
6. ordino il risultato

7. Aggregazioni

Le aggregazioni producono sintesi.

```
SELECT e.nome_esercizio, COUNT(ex.id_esecuzione) AS numero_esecuzioni
FROM Esercizi e
LEFT JOIN SchedaEsercizi se ON e.id_esercizio = se.id_esercizio
LEFT JOIN Esecuzioni ex ON se.id_scheda = ex.id_scheda
    AND se.ordine_esecuzione = ex.ordine_esecuzione
GROUP BY e.id_esercizio, e.nome_esercizio
ORDER BY numero_esecuzioni DESC;
```

Da osservare:

- COUNT(ex.id_esecuzione) conta solo valori non nulli
- LEFT JOIN mantiene anche esercizi mai eseguiti

- GROUP BY definisce i gruppi

8. HAVING

WHERE filtra righe prima del raggruppamento.

HAVING filtra gruppi dopo il raggruppamento.

```
SELECT e.nome_esercizio, COUNT(ex.id_esecuzione) AS numero_esecuzioni
FROM Esercizi e
LEFT JOIN SchedaEsercizi se ON e.id_esercizio = se.id_esercizio
LEFT JOIN Esecuzioni ex ON se.id_scheda = ex.id_scheda
    AND se.ordine_esecuzione = ex.ordine_esecuzione
GROUP BY e.id_esercizio, e.nome_esercizio
HAVING COUNT(ex.id_esecuzione) >= 2;
```

9. Select innestate

Una select innestata e' una query usata dentro un'altra query.

Filtro con IN

```
SELECT nome_esercizio, categoria
FROM Esercizi
WHERE categoria IN (
    SELECT categoria
    FROM Esercizi
    WHERE nome_esercizio IN ('Squat', 'Affondi')
);
```

La query interna produce un insieme di categorie. La query esterna usa quell'insieme per filtrare.

Confronto con una soglia calcolata

```
SELECT i.nome, i.cognome, COUNT(ex.id_esecuzione) AS numero_esecuzioni
FROM Iscritti i
LEFT JOIN Esecuzioni ex ON i.id_iscritto = ex.id_iscritto
GROUP BY i.id_iscritto, i.nome, i.cognome
HAVING COUNT(ex.id_esecuzione) >= (
    SELECT AVG(conteggio)
    FROM (
        SELECT COUNT(*) AS conteggio
        FROM Esecuzioni
        GROUP BY id_iscritto
    )
);
```

Qui la soglia non e' scritta a mano. Viene calcolata da una query interna.

10. Transazioni: idea generale

Una transazione e' un blocco di operazioni che deve essere trattato come un'unita' logica.

Esempio applicativo:

- creare una nuova scheda
- inserire gli esercizi della scheda
- verificare che tutti i riferimenti siano validi
- rendere visibile il risultato solo se tutto e' andato bene

Se una delle operazioni fallisce, vogliamo annullare anche le precedenti.

11. Perché servono le transazioni

Senza transazioni, una sequenza di istruzioni puo' lasciare il database in uno stato intermedio.

Esempio:

1. creo una nuova scheda
2. inserisco il primo esercizio
3. il secondo esercizio fallisce per chiave esterna non valida

Risultato indesiderato:

- la scheda esiste
- ma e' incompleta

Una transazione permette di evitare questo stato.

12. Comandi transazionali principali

BEGIN

Avvia una transazione esplicita.

BEGIN;

COMMIT

Rende definitive le modifiche.

COMMIT;

ROLLBACK

Annulla le modifiche dall'inizio della transazione.

ROLLBACK;

SAVEPOINT

Definisce un punto intermedio a cui e' possibile tornare.

```
SAVEPOINT punto_intermedio;
```

ROLLBACK TO

Annulla le modifiche fino a un savepoint.

```
ROLLBACK TO punto_intermedio;
```

RELEASE

Chiude un savepoint.

```
RELEASE punto_intermedio;
```

13. Esempio con ROLLBACK

```
BEGIN;
```

```
INSERT INTO Contatti (id, nome, cognome, telefono)
VALUES (6, 'Elena', 'Bruni', '06-0000000');
```

```
UPDATE Contatti
SET telefono = '06-1111111'
WHERE id = 1;
```

```
ROLLBACK;
```

Dopo ROLLBACK, nessuna delle due modifiche resta nello stato finale.

14. Esempio con COMMIT

```
BEGIN;
```

```
INSERT INTO Contatti (id, nome, cognome, telefono)
VALUES (6, 'Elena', 'Bruni', '06-0000000');
```

```
COMMIT;
```

Dopo COMMIT, la modifica diventa parte dello stato confermato.

15. Esempio con SAVEPOINT

```
BEGIN;
```

```
INSERT INTO Esecuzioni (...)
VALUES (...);
```



```
SAVEPOINT seconda_esecuzione;
```

```
INSERT INTO Esecuzioni (...)  
VALUES (...);
```

```
ROLLBACK TO seconda_esecuzione;  
RELEASE seconda_esecuzione;
```

```
COMMIT;
```

Il primo inserimento resta, il secondo viene annullato.

16. Transazioni e vincoli

Le transazioni non sostituiscono i vincoli.

Lavorano insieme:

- i vincoli dicono quali stati sono ammessi
- la transazione decide se confermare o annullare un passaggio di stato

Una buona applicazione usa entrambi.

17. Errori tipici

- dimenticare WHERE in UPDATE o DELETE
- usare JOIN quando serve LEFT JOIN
- filtrare gruppi con WHERE invece che con HAVING
- scrivere query innestate senza capire quale insieme producono
- usare transazioni solo dopo aver già modificato dati importanti
- confondere ROLLBACK con una cancellazione manuale

18. Metodo di lettura di una query

Per leggere una query complessa:

1. individuare le tabelle
2. individuare i join
3. individuare i filtri sulle righe
4. individuare eventuali raggruppamenti
5. individuare filtri sui gruppi
6. individuare select innestate
7. leggere il risultato atteso in linguaggio naturale

19. Collegamento con i laboratori

Nei laboratori REPL e web, ogni operazione applicativa corrisponde a una o più istruzioni SQL.

Esempi:

- creazione iscritto: `INSERT`
- aggiornamento scheda: `UPDATE`
- cancellazione esercizio: `DELETE` bloccabile da `FOREIGN KEY`
- dettaglio scheda: `JOIN` multipli
- storico esecuzioni: `JOIN`, `WHERE`, `ORDER BY`

Quando un'operazione applicativa richiede piu' istruzioni, la transazione diventa lo strumento naturale.

20. Risultato didattico atteso

Alla fine della lezione lo studente dovrebbe saper:

- classificare i principali comandi SQL
- leggere CRUD su schema e CRUD su dati
- interpretare join, filtri e query innestate
- distinguere `WHERE` e `HAVING`
- spiegare il ruolo di `BEGIN`, `COMMIT`, `ROLLBACK`, `SAVEPOINT`
- collegare SQL alle operazioni dei laboratori applicativi